

Student Portal — Google Sheets Mapping & Admin Manual

Covers `pages/api/portal-config.js` and `pages/portal.js`. Last updated for the "Class Levels" change (v3) that moves `ClassLevel` out of hardcoded JS into a sheet, and extends it to 12 levels: Class 0-10 + Adult.

1. How this system works (quick overview)

There is one Master Config Sheet plus one Sheet per subject.

```
MASTER CONFIG SHEET (Sheet ID hardcoded as MASTER_SHEET_ID in portal-
config.js)
├ "Class Levels"          ← NEW — defines every class (0-10 + Adult)
├ "Class Subject Map"     ← which subjects show to which class, icons/colors
├ "Subject Sheet Map"     ← legacy, fallback only
└ "Assignment Subjects"  ← legacy, fallback only

PER-SUBJECT SHEET (one per row in "Class Subject Map", linked by Sheet ID)
├ "Learning Modules"      ← chapters/topics for that subject
└ "Learning Steps"        ← quiz steps within each module
```

The Next.js API route `pages/api/portal-config.js` fetches all of this via CSV export (`gviz/tq?tqx=out:csv`), filters it by the student's class, caches it for 60 seconds, and returns one JSON payload to `portal.js`. **Nothing about subjects, chapters, or classes is hardcoded in the app anymore** — it's all editable by an admin directly in the Sheets.

⚠ Every sheet must be shared as "Anyone with the link → Viewer", or the fetch will silently fail (the API logs a warning and falls back to defaults).

2. NEW: "Class Levels" tab — manage classes here

Location: a tab named exactly `Class Levels` inside the Master Config Sheet (`MASTER_SHEET_ID` in `portal-config.js`).

This is the tab you asked for — it replaces the old hardcoded `Class 1`...`Class 5` list and the `CLASS_TO_AGE` map that used to live in `portal.js`.

Columns (row 1 headers — must match exactly)

Column	Required?	Type	Description
<code>Class</code>	Yes	Text	Canonical ID. Must exactly match the <code>Class</code> column used in "Class Subject Map", "Learning Modules", and

Column	Required?	Type	Description
			"Learning Steps". This is the value stored on a student's profile.
Label	No	Text	Friendly display name shown in the UI (pills, dropdown). Defaults to Class if left blank.
Age	No	Text	Short age-group text shown next to the class badge, e.g. 8-9 yrs , 18+ yrs .
Aliases	No	Text	Comma-separated alternate values that should resolve to this class, e.g. kg, nursery, 0 . Used when matching the ?classLevel= query param or values coming from your student database.
Display Order	No	Number	Sort order for pills/dropdown (lower = first).
Active	No	TRUE/FALSE	Uncheck to hide a class everywhere without deleting the row.

Rows to create (0-10 + Adult — 12 total)

Class	Label	Age	Aliases	Display Order	Active
Class 0	Class 0 (Nursery/KG)	4-5 yrs	kg, nursery, 0	1	TRUE
Class 1	Class 1	5-6 yrs	1	2	TRUE
Class 2	Class 2	6-7 yrs	2	3	TRUE
Class 3	Class 3	7-8 yrs	3	4	TRUE
Class 4	Class 4	8-9 yrs	4	5	TRUE
Class 5	Class 5	9-10 yrs	5	6	TRUE
Class 6	Class 6	10-11 yrs	6	7	TRUE
Class 7	Class 7	11-12 yrs	7	8	TRUE
Class 8	Class 8	12-13 yrs	8	9	TRUE
Class 9	Class 9	13-14 yrs	9	10	TRUE
Class 10	Class 10	14-15 yrs	10	11	TRUE
Adult	Adult	18+ yrs	adults, adult	12	TRUE

Feel free to edit `Label` and `Age` to whatever wording you like (e.g. rename "Class 0" to "Nursery", or "Adult" to "Adult Learner") — the `Class` value in the first column must stay the same once content is tagged against it, since that's the key everything else joins on.

What happens if you don't create this tab yet

The code ships with a safe built-in default of these same 12 rows, so nothing breaks. Once you add the "Class Levels" tab with at least one row, the Sheet takes over completely.

How to add a 13th class later (e.g. "Class 11")

1. Add a row to "Class Levels" with `Class = Class 11`.
2. Tag any "Class Subject Map", "Learning Modules", or "Learning Steps" rows for that class with `Class 11` in their `Class` column.
3. Done — no code or deploy needed.

3. "Class Subject Map" tab — which subjects show to which class

Same Master Config Sheet, tab `Class Subject Map`.

Column	Description
<code>Subject</code>	Subject name shown on the card, e.g. <code>Mathematics</code> .
<code>Class</code>	Must match a <code>Class</code> value from "Class Levels". Blank = shown to all classes.
<code>Sheet ID</code>	The Google Sheet ID of that subject's own sheet (see §4 below).
<code>Tagline</code>	Short subtitle shown on the subject card.
<code>Icon (FontAwesome solid)</code>	e.g. <code>fa-square-root-variable</code> . Defaults to <code>fa-book</code> .
<code>Emoji</code>	Optional emoji shown alongside the icon.
<code>Color (Hex)</code>	Card accent color, e.g. <code>#f97316</code> . Defaults to <code>#00c6a7</code> .
<code>Display Order</code>	Sort order for subject cards.
<code>Active</code>	TRUE/FALSE.

Multiple rows can point at the same `Sheet ID` (e.g. one row per class for "Mathematics") — the API only fetches each unique Sheet ID once.

4. Per-subject sheet — "Learning Modules" tab

One tab per subject sheet (the Sheet ID comes from "Class Subject Map" above).

Column	Description
Module ID	Unique ID, e.g. LM01.
Title	Topic/chapter title.
Subject	Must exactly match the Subject value in "Class Subject Map".
Class	Must match a Class value from "Class Levels". Blank = visible to all classes.
Icon (FontAwesome solid), Emoji, Color (Hex)	Display styling.
Introduction	Intro paragraph shown when a student opens the topic.
Now You Try	Optional practice prompt.
Intro Image URL	Optional image.
Subtopics	Optional comma-separated pill list shown on the card.
Display Order	Sort order.
Active	TRUE/FALSE.

5. Per-subject sheet — "Learning Steps" tab

Column	Description
Module ID	Links back to a row in "Learning Modules".
Step Number	Order within the module.
Class	Same convention as above (denormalised here to avoid a join at render time).
(quiz/step content columns — question, options, answer, etc.)	Specific to your quiz format; not changed by this update.
Active	TRUE/FALSE.

6. Where `classLevel` flows through the code

1. Student logs in (or sets their class in **Profile**) → `profile.classLevel` is set to one of the canonical `Class` values from "Class Levels" (e.g. `Class 7`, `Adult`).
 2. `portal.js` calls `GET /api/portal-config?classLevel=Class 7`.
 3. `portal-config.js`:
 - Fetches "Class Levels" → normalises the incoming value to a canonical `Class`.
 - Fetches "Class Subject Map", "Learning Modules", "Learning Steps" and keeps only rows where `Class` matches (or is blank).
 - Returns `{ assignmentSubjects, learningModules, learningSteps, classLevels }`.
 4. `portal.js` renders:
 - The "Quick Set" class picker (onboarding) and the **My Class / Class pills** filter row — both now generated from `cfg.classLevels` instead of a hardcoded array.
 - The **Profile → Class Level** dropdown — same source.
-

7. Things this update fixed along the way

While wiring this up, two pre-existing bugs in `portal.js` were also fixed so the file actually compiles and runs:

- A duplicate `const studentAgeGroup` declaration (would throw a `SyntaxError` at build time).
 - A broken "Quick Set" class-picker block that referenced undefined `curriculum`/`CMS` variables and had dead leftover code mixed into a stray button — replaced with a single clean map over the sheet-driven class list.
-

8. Out of scope — not covered by the two files reviewed

`portal.js` also calls these API routes, which were **not** included in the files you uploaded, so their underlying Sheets/columns aren't documented here:

- `/api/student/auth`, `/api/student/profile` — login + profile save (this is almost certainly backed by the Apps Script you linked:
`https://script.google.com/macros/s/AKfycbz.../exec`)).
- `/api/student/subjects`, `/api/student/assignments`, `/api/student/attendance`,
`/api/student/progress`, `/api/student/progress-history`, `/api/student/leaderboard`.

Important consistency note: wherever your student database (behind that Apps Script) stores a class value for each student, it must use the **exact same canonical** `Class` strings as the new "Class Levels" tab (`Class 0` ... `Class 10`, `Adult`) — or add the old/raw values to that student's row to the `Aliases` column in "Class Levels" so they still resolve correctly.

If you share those API route files (or the Apps Script project), I can extend this manual to cover them too.

9. Files changed

- `pages/api/portal-config.js` — added "Class Levels" tab fetch, dynamic class normalization (replacing the hardcoded `Class 1`–`Class 5` list), and `classLevels` in the API response.
- `pages/portal.js` — consumes `cfg.classLevels` everywhere a class list/dropdown is rendered; fallback data extended to all 12 classes; two pre-existing bugs fixed.